

VS - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continuous color palettes

# load needed functions
# source(paste0(RootDir, "/src/function_library/sp_functions.R"))
# do this in the calling R script

# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste(CurPat, " - ", CurTask, " - ", RunLabel, " - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE

CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres

# done in calling script
# print(paste0("Using root dir: ", RootDir))
# RunDir <- paste0(paste0(RootDir, "/output/"), RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir, showWarnings = TRUE)
#   print(paste0("created run directory: ", RunDir))
# }
```

```

# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir, showWarnings = TRUE)
#   print(paste0("created participant directory: ", PatientDir))
# }
# TablesDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir, showWarnings = TRUE)
#   print(paste0("created tables directory: ", TablesDir))
# }
# FigDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir, showWarnings = TRUE)
#   print(paste0("created figures directory: ", FigDir))
# }
# ReportsDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir, showWarnings = TRUE)
#   print(paste0("created reports directory: ", ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())

ModelDatFilename<-paste0(RootDir, "/data/", CurPat, "/", CurPat, "_", CurTask, "_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ", ModelDatFilename))
  print(sprintf("\troot dir: ", RootDir))
}
PosDat<-read.csv(ModelDatFilename)

# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values", OrigDataLen))

## [1] "Original data has 4079 values"

if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values", DataLen))
  print(sprintf("%d values eliminated.", OrigDataLen - DataLen))
}

```

```
## [1] "Data without fractional preserved values has 3797 values"
## [1] "282 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite
```

```
# get counts of syllable components in different serial positions
```

```
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syll_component))
```

```
syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))
```

```
kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	501	33	120	NA	NA	654
2	58	NA	380	89	103	630
3	276	NA	156	176	16	624
4	263	NA	199	64	35	561
5	204	NA	171	63	34	472
6	168	NA	115	63	20	366
7	154	NA	78	27	16	275
8	80	NA	37	22	4	143
9	65	NA	1	NA	6	72

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7660550	0.0504587	0.1834862	NA	NA	654
2	0.0920635	NA	0.6031746	0.1412698	0.1634921	630
3	0.4423077	NA	0.2500000	0.2820513	0.0256410	624
4	0.4688057	NA	0.3547237	0.1140820	0.0623886	561
5	0.4322034	NA	0.3622881	0.1334746	0.0720339	472
6	0.4590164	NA	0.3142077	0.1721311	0.0546448	366
7	0.5600000	NA	0.2836364	0.0981818	0.0581818	275
8	0.5594406	NA	0.2587413	0.1538462	0.0279720	143
9	0.9027778	NA	0.0138889	NA	0.0833333	72

```
# get percentages only from summary table
```

```
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
```

```
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
```

```
syll_comp_perc <- syll_comp_perc %>% rename("Coda" = "1", "Satellite" = "S")
```

```
# pivot to long format for plotting
```

```
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
  names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda", "Satellite"))
```

```
# plot satellite and coda occurrences across position

syll_comp_plot <- ggplot(syll_comp_plot_data,
                        aes(x=pos,y=percent,group=syll_component,
                           color=syll_component,
                           linetype = syll_component,
                           shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype_manual(name="Syllable component", values = palette_linetypes) +
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_pos_of_syll_complexities_in_stimuli.png"), plot=syll_comp_plot)
```

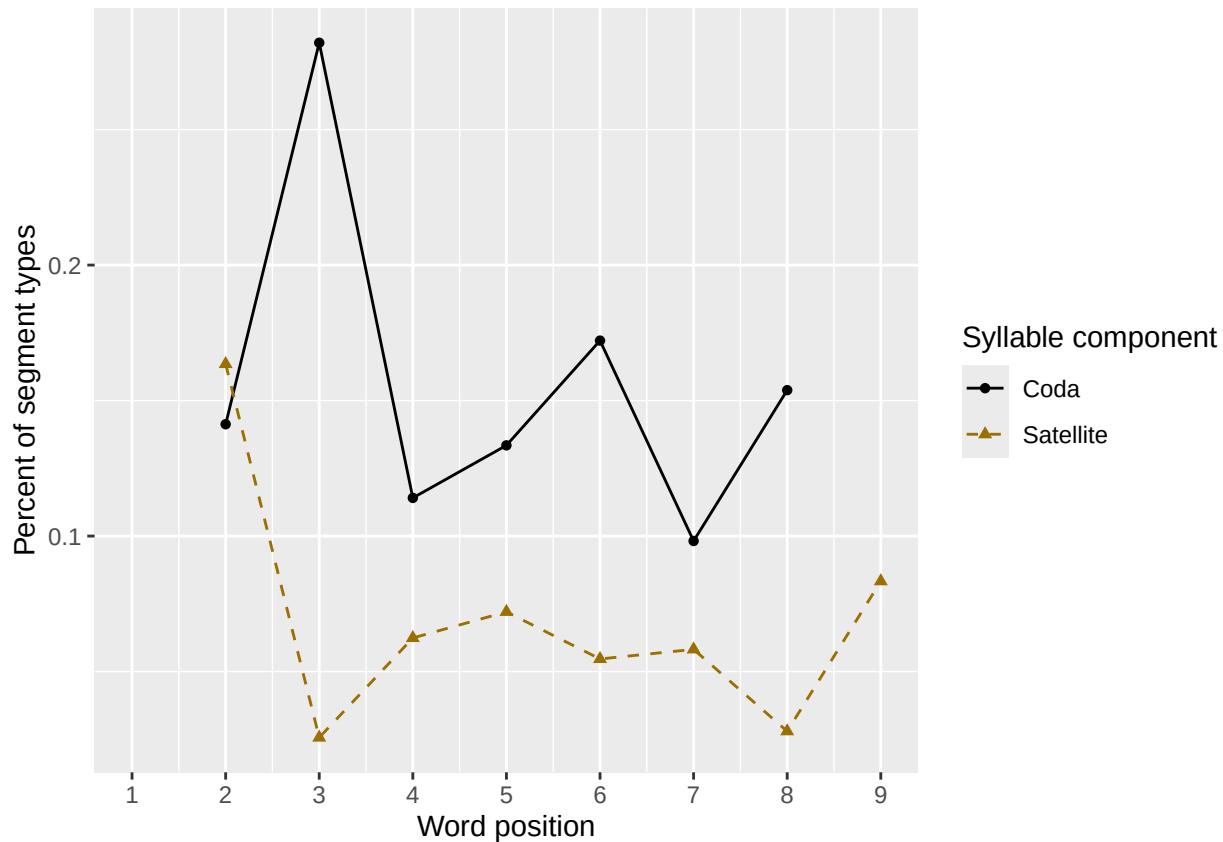
Warning: Removed 3 rows containing missing values or values outside the scale range (`geom\_line()`).

Warning: Removed 3 rows containing missing values or values outside the scale range (`geom\_point()`).

syll_comp_plot

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

Removed 3 rows containing missing values or values outside the scale range (`geom\_point()`).



```
# len/pos table
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```

min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask, "_preserved_percentages_by_len",
pos_len_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1     4 0.929 0.964 0.821 NA      NA      NA      NA      NA      NA
## 2     5 0.954 0.860 0.819 0.824 NA      NA      NA      NA      NA
## 3     6 0.938 0.849 0.785 0.686 0.677 NA      NA      NA      NA
## 4     7 0.92  0.887 0.854 0.75  0.698 0.588 NA      NA      NA
## 5     8 0.930 0.862 0.822 0.705 0.636 0.594 0.526 NA      NA
## 6     9 0.892 0.873 0.785 0.675 0.744 0.558 0.413 0.407 NA
## 7    10 0.945 0.956 0.794 0.615 0.493 0.441 0.365 0.452 0.403

```

```

# len/pos table
pos_len_N <- PosDat %>% group_by(stimlen, pos) %>% summarise(preserved = mean(preserved), N=n()) %>% dplyr::

```

`summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```

pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
"_N_by_len_position.csv"))
pos_len_N_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1     4    56    56    56    NA    NA    NA    NA    NA    NA
## 2     5    87    86    83    85    NA    NA    NA    NA    NA
## 3     6   113   106   107   102    99    NA    NA    NA    NA
## 4     7   100    97    96   100    96    97    NA    NA    NA
## 5     8   142   138   135   132   132   133   137    NA    NA
## 6     9    83    79    79    77    78    77    75    81    NA
## 7    10    73    68    68    65    67    59    63    62    72

```

```

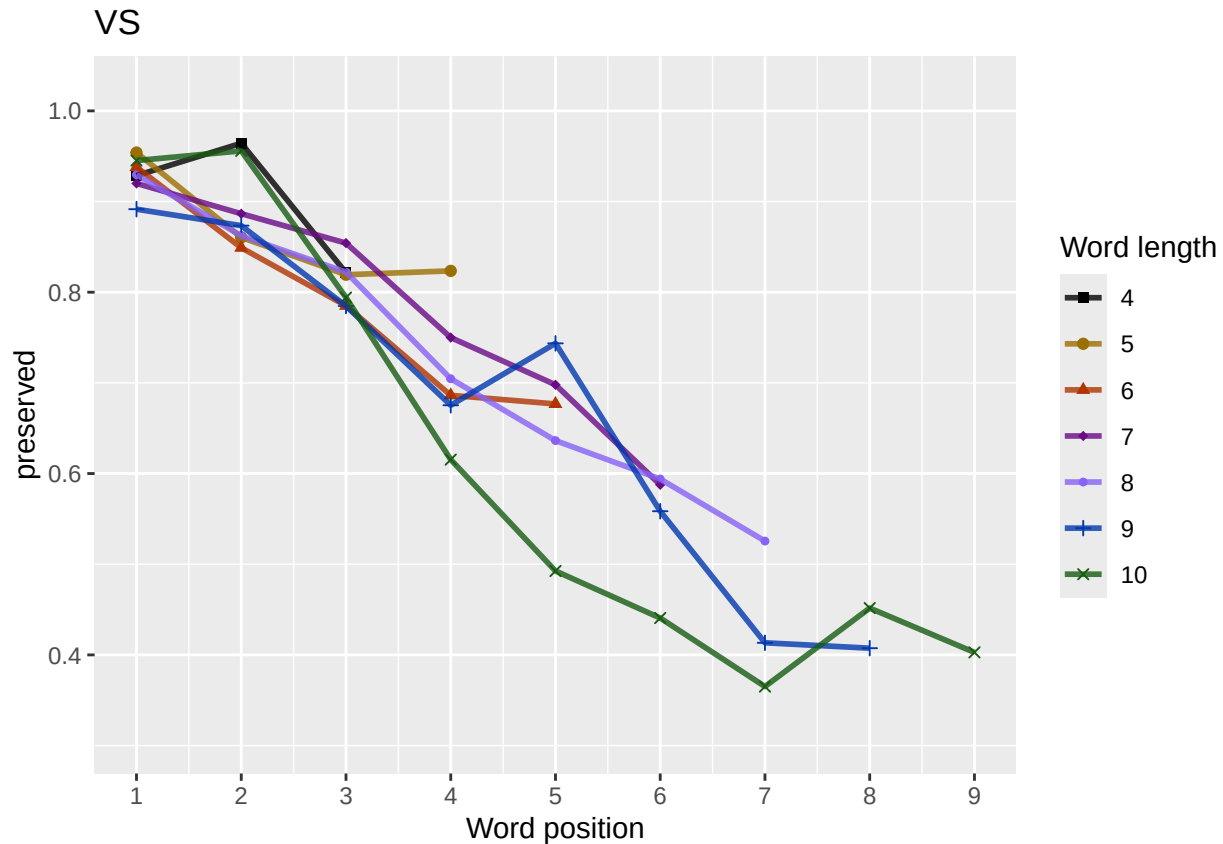
obs_linetypes <- c("solid", "solid", "solid", "solid",
"solid", "solid", "solid", "solid", "solid")
pred_linetypes <- "dashed"
pos_len_summary$stimlen <- factor(pos_len_summary$stimlen)
pos_len_summary$pos <- factor(pos_len_summary$pos)
# len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
paste0(CurPat),
NULL,
c(min_preserved, max_preserved),
palette_values,
shape_values,
obs_linetypes,
pred_linetypes)

```

`summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot, device="png", unit="cm", width=15, height=11)
len_pos_plot
```



Length and position

```
# length and position
```

```
LPMModelEquations<-c("preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",
  "preserved ~ stimlen*pos",
  "preserved ~ I(pos^2)+pos",
  "preserved ~ stimlen + I(pos^2) + pos",
  "preserved ~ stimlen * (I(pos^2) + pos)"
)
```

```
LPres<-TestModels(LPMModelEquations,PosDat)
```

```
## *****
```

```
## model index: 8
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```

##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stimlen:I(pos^2)
##      2.847382      0.069574      0.033602      -0.406126      0.001867      -0.001867
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3791 Residual
## Null Deviance:      4368
## Residual Deviance: 3830 AIC: 3842
## log likelihood: -1915.226
## Nagelkerke R2: 0.1931346
## % pres/err predicted correctly: -1263.236
## % of predictable range [ (model-null)/(1-null) ]: 0.1391104
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)      pos
##      3.87402      -0.08311      0.03588      -0.71173
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3793 Residual
## Null Deviance:      4368
## Residual Deviance: 3835 AIC: 3843
## log likelihood: -1917.441
## Nagelkerke R2: 0.1916519
## % pres/err predicted correctly: -1265.45
## % of predictable range [ (model-null)/(1-null) ]: 0.1376027
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      3.27014      0.03142      -0.69810
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3794 Residual
## Null Deviance:      4368
## Residual Deviance: 3844 AIC: 3850
## log likelihood: -1921.796
## Nagelkerke R2: 0.1887317
## % pres/err predicted correctly: -1269.173
## % of predictable range [ (model-null)/(1-null) ]: 0.1350676
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      pos

```

```

##      3.11709      -0.06222      -0.38525
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3794 Residual
## Null Deviance:      4368
## Residual Deviance: 3852 AIC: 3858
## log likelihood: -1926.024
## Nagelkerke R2: 0.1858904
## % pres/err predicted correctly: -1266.877
## % of predictable range [ (model-null)/(1-null) ]: 0.1366311
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      pos      stimlen:pos
##      3.46950      -0.10490      -0.48421      0.01166
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3793 Residual
## Null Deviance:      4368
## Residual Deviance: 3851 AIC: 3859
## log likelihood: -1925.645
## Nagelkerke R2: 0.1861454
## % pres/err predicted correctly: -1267.405
## % of predictable range [ (model-null)/(1-null) ]: 0.1362715
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      2.723      -0.406
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3795 Residual
## Null Deviance:      4368
## Residual Deviance: 3857 AIC: 3861
## log likelihood: -1928.587
## Nagelkerke R2: 0.1841652
## % pres/err predicted correctly: -1269.561
## % of predictable range [ (model-null)/(1-null) ]: 0.1348031
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      3.0699      -0.2615
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3795 Residual

```



```
## Null Deviance:      4368
## Residual Deviance: 4237 AIC: 4241
## log likelihood: -2118.599
## Nagelkerke R2: 0.0495386
## % pres/err predicted correctly: -1418.497
## % of predictable range [ (model-null)/(1-null) ]: 0.03338451
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 1.035
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3796 Residual
## Null Deviance:      4368
## Residual Deviance: 4368 AIC: 4370
## log likelihood: -2183.993
## Nagelkerke R2: 0
## % pres/err predicted correctly: -1467.523
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                          AIC=LPRes$AIC,
                          row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                             "_len_pos_model_summary.csv"),row.names = FALSE)

kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	3842.451	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	-	-	0.0336007	0.0018668
preserved ~ stimlen + I(pos^2) + pos	3842.882	0.4307588	0.8062355	0.4393403	0.1916513	0.874024	-	-	NA	0.0358834	NA
preserved ~ I(pos^2) + pos	3849.597	7.1414026	0.0281361	0.1015332	0.1887313	0.270144	NA	-	NA	0.0314162	NA
preserved ~ stimlen + pos	3858.045	15.5975967	0.0004102	0.0002235	0.1858964	0.117087	-	-	NA	NA	NA

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * pos	3859.2916	6.839506	0.002205	0.000120	11861434	469498	-	-	0.0116603	NA
preserved ~ pos	3861.1748	7.230588	0.000860	0.000046	1841632	272653	NA	-	NA	NA
preserved ~ stimlen	4241.1938	8.746706	0.000000	0.000000	0495386	69940	-	NA	NA	NA
preserved ~ 1	4369.9872	7.535463	0.000000	0.000000	0000000	35346	NA	NA	NA	NA

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ stimlen * (I(pos^2) + pos)"
```

```
print(BestLPModel)
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```
##
```

```
## Coefficients:
```

```
## (Intercept) stimlen I(pos^2) pos stimlen:I(pos^2) stimlen:I(pos^2)
```

```
## 2.847382 0.069574 0.033602 -0.406126 0.001867 -0.001867
```

```
##
```

```
## Degrees of Freedom: 3796 Total (i.e. Null); 3791 Residual
```

```
## Null Deviance: 4368
```

```
## Residual Deviance: 3830 AIC: 3842
```

```
PosDat$LPFitted<-fitted(BestLPModel)
```

```
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],  
NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)
```

```
fitted_pos_len_table
```

```
## # A tibble: 7 x 10
```

```
## # Groups: stimlen [7]
```

```
## stimlen `1` `2` `3` `4` `5` `6` `7` `8` `9`
```

```
## <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
```

```
## 1 4 0.928 0.888 0.841 NA NA NA NA NA NA
```

```
## 2 5 0.929 0.885 0.832 0.775 NA NA NA NA NA
```

```
## 3 6 0.931 0.883 0.822 0.756 0.695 NA NA NA NA
```

```
## 4 7 0.932 0.880 0.812 0.736 0.664 0.607 NA NA NA
```

```
## 5 8 0.933 0.877 0.802 0.715 0.633 0.565 0.520 NA NA
```

```
## 6 9 0.935 0.875 0.791 0.694 0.600 0.523 0.471 0.444 NA
```

```
## 7 10 0.936 0.872 0.780 0.671 0.566 0.481 0.422 0.390 0.384
```

```
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
```

```
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
```

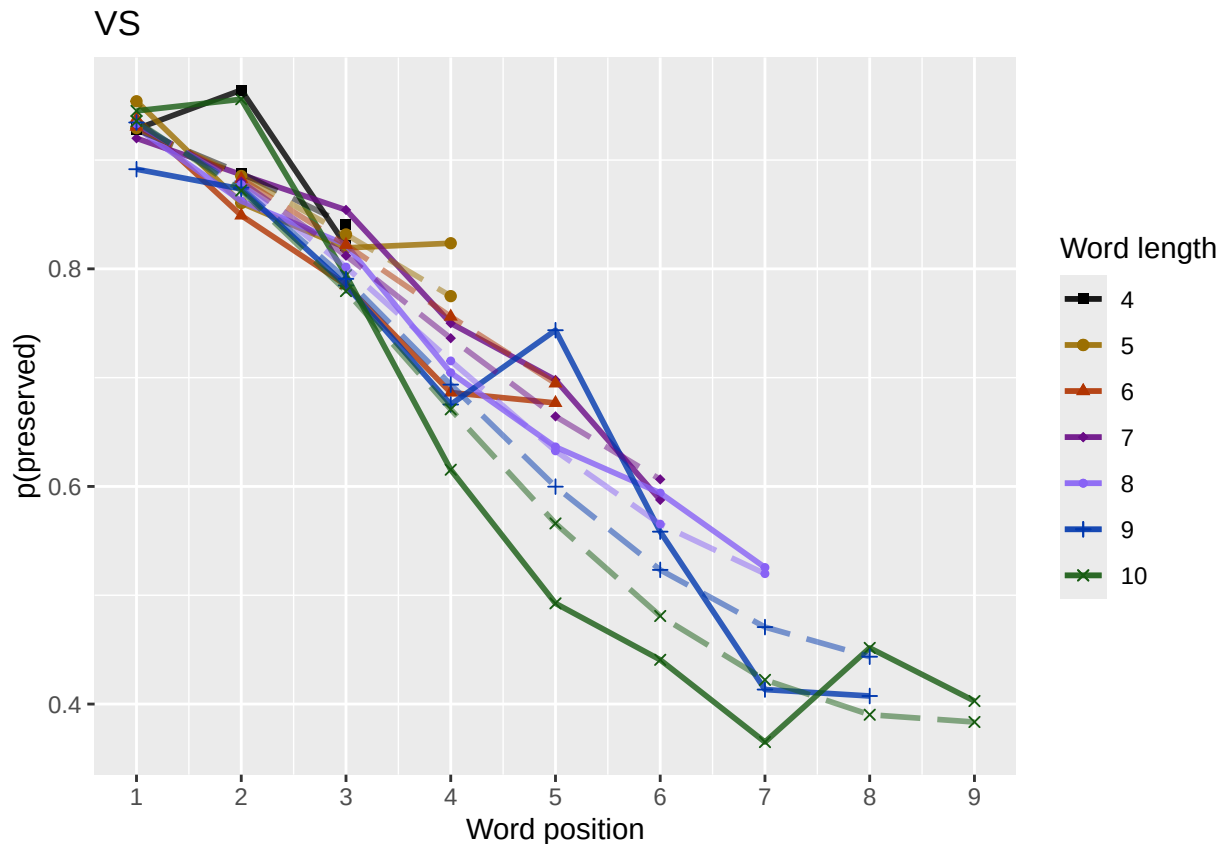
```

# fitted_len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos, y=fitted))
# geom_point(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen)) + ggtitle(paste0(
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
  paste0(PosDat$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_percent_preserved_by_length_pos_wfit.png"), plot=fitted_len_pos_plot

```



length and position without fragments to see if this changes position² influence

```

# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models

# number responses
resp_num <- 0

```

```

prev_pos<-9999 # big number to initialize (so first position is smaller)
resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array

# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag

## # A tibble: 1 x 2
##   frag_sum      N
##   <int> <int>
## 1      74   665

num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100

## Warning: Unknown or uninitialised column: `percent_with_frag`.

print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
  num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))

## [1] "The number of responses with fragments was 74 / 665 = 11.13 percent"

write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
  CurTask,"_percent_fragments.csv"),row.names = FALSE)

# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)

## *****
## model index: 8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##   data = PosDat)
##
## Coefficients:
##   (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stimlen
##   2.688735      0.087095      0.038719     -0.307993      0.002288      -0.000000
##
## Degrees of Freedom: 3636 Total (i.e. Null); 3631 Residual
## Null Deviance:      3919
## Residual Deviance: 3548 AIC: 3560
## log likelihood: -1773.987
## Nagelkerke R2: 0.1470563
## % pres/err predicted correctly: -1155.354
## % of predictable range [ (model-null)/(1-null) ]: 0.1012281
## *****

```

```

## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##      3.96659      -0.10190      0.04122      -0.69518
##
## Degrees of Freedom: 3636 Total (i.e. Null); 3633 Residual
## Null Deviance:      3919
## Residual Deviance: 3555 AIC: 3563
## log likelihood: -1777.313
## Nagelkerke R2: 0.1445503
## % pres/err predicted correctly: -1158.562
## % of predictable range [ (model-null)/(1-null) ]: 0.09873448
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      3.22355      0.03583      -0.67882
##
## Degrees of Freedom: 3636 Total (i.e. Null); 3634 Residual
## Null Deviance:      3919
## Residual Deviance: 3567 AIC: 3573
## log likelihood: -1783.476
## Nagelkerke R2: 0.1398939
## % pres/err predicted correctly: -1163.564
## % of predictable range [ (model-null)/(1-null) ]: 0.09484669
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      3.12709      -0.07885      -0.32475
##
## Degrees of Freedom: 3636 Total (i.e. Null); 3634 Residual
## Null Deviance:      3919
## Residual Deviance: 3575 AIC: 3581
## log likelihood: -1787.693
## Nagelkerke R2: 0.1366986
## % pres/err predicted correctly: -1161.608
## % of predictable range [ (model-null)/(1-null) ]: 0.09636729
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##      3.409418    -0.112994    -0.406418      0.009613
##
## Degrees of Freedom: 3636 Total (i.e. Null);  3633 Residual
## Null Deviance:      3919
## Residual Deviance: 3575  AIC: 3583
## log likelihood:  -1787.457
## Nagelkerke R2:  0.1368775
## % pres/err predicted correctly:  -1162.004
## % of predictable range [ (model-null)/(1-null) ]:  0.09605957
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      2.6218      -0.3502
##
## Degrees of Freedom: 3636 Total (i.e. Null);  3635 Residual
## Null Deviance:      3919
## Residual Deviance: 3583  AIC: 3587
## log likelihood:  -1791.554
## Nagelkerke R2:  0.1337672
## % pres/err predicted correctly:  -1165.146
## % of predictable range [ (model-null)/(1-null) ]:  0.09361692
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      3.055      -0.238
##
## Degrees of Freedom: 3636 Total (i.e. Null);  3635 Residual
## Null Deviance:      3919
## Residual Deviance: 3823  AIC: 3827
## log likelihood:  -1911.518
## Nagelkerke R2:  0.03949837
## % pres/err predicted correctly:  -1252.591
## % of predictable range [ (model-null)/(1-null) ]:  0.02565068
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:

```

```
## (Intercept)
##      1.211
##
## Degrees of Freedom: 3636 Total (i.e. Null); 3636 Residual
## Null Deviance:      3919
## Residual Deviance: 3919 AIC: 3921
## log likelihood: -1959.522
## Nagelkerke R2: 0
## % pres/err predicted correctly: -1285.593
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****

NoFragBestLPModel<-NoFrag_LPRes$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPRes$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPRes$Model,
                                AIC=NoFrag_LPRes$AIC,
                                row.names = NoFrag_LPRes$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPRes$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPRes$CoefficientValues, by='row.names',sort=FALSE)
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	3559.974	0.000000	1.000000	0.0789156	0.1470523	36887350.0870952	-	-	0.0387188	0.0022879
preserved ~ stimlen + I(pos^2) + pos	3562.622	2.651362	0.265622	0.209617	0.2144553	3966589	-	-	NA	0.0412165
preserved ~ I(pos^2) + pos	3572.952	12.977538	0.001520	0.001199	0.8139893	3223554	NA	-	NA	0.0358259
preserved ~ stimlen + pos	3581.382	21.412142	0.000022	0.000017	0.7136698	6127093	-	-	NA	NA
preserved ~ stimlen * pos	3582.912	22.940331	0.000010	0.000008	0.2136873	5409418	-	-	0.0096129	NA
preserved ~ pos	3587.107	7.132736	0.000000	0.000000	0.0133762	2621759	NA	-	NA	NA
preserved ~ stimlen	3827.032	267.062034	0.000000	0.000000	0.0039498	4054680	-	NA	NA	NA
preserved ~ 1	3921.043	361.069179	0.000000	0.000000	0.000000	0210657	NA	NA	NA	NA

```
# plot no fragment data
```

```
NoFragData$LPFitted<-fitted(NoFragBestLPModel)
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
  NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

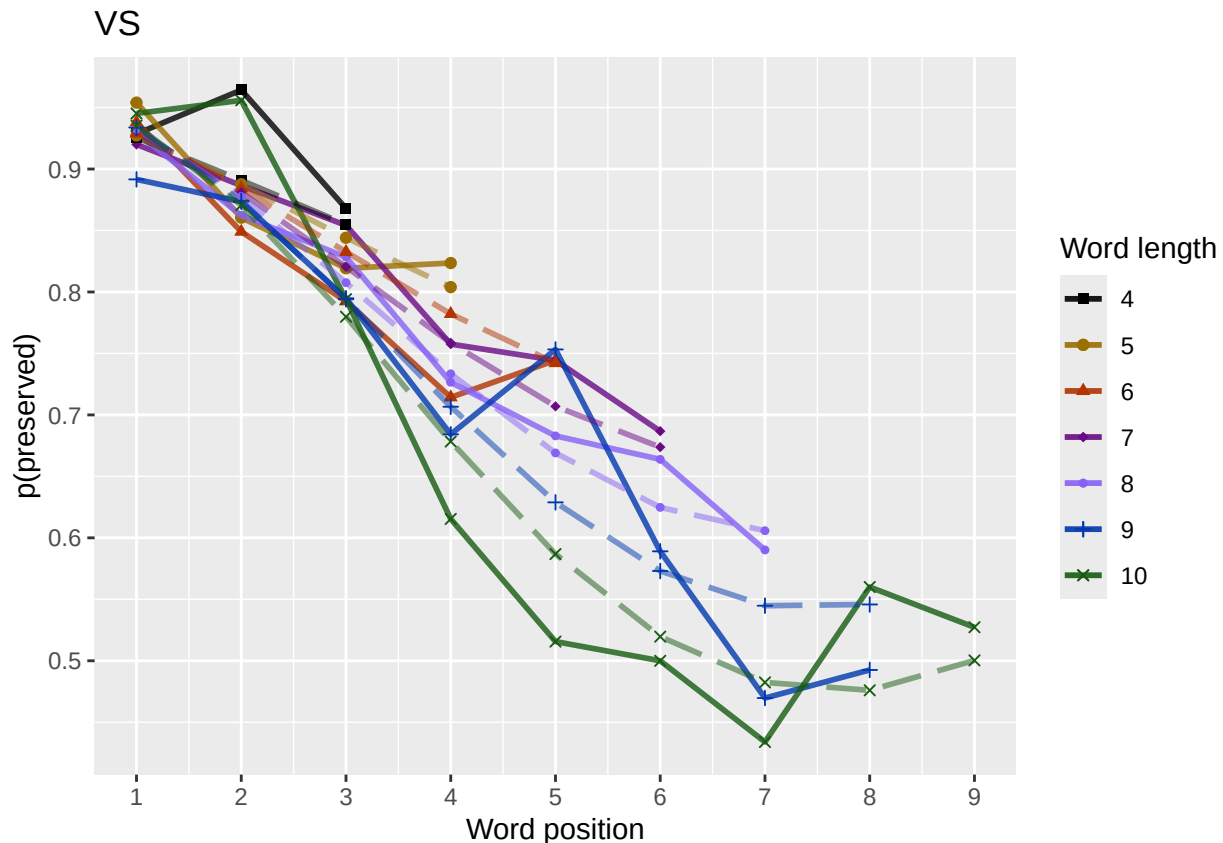
```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.926 0.891 0.855 NA      NA      NA      NA      NA      NA
## 2      5 0.927 0.888 0.844 0.804 NA      NA      NA      NA      NA
## 3      6 0.929 0.884 0.833 0.782 0.742 NA      NA      NA      NA
## 4      7 0.931 0.881 0.820 0.759 0.707 0.674 NA      NA      NA
## 5      8 0.932 0.878 0.808 0.733 0.669 0.625 0.606 NA      NA
## 6      9 0.934 0.874 0.794 0.707 0.629 0.573 0.545 0.546 NA
## 7     10 0.935 0.871 0.780 0.678 0.587 0.520 0.482 0.476 0.500
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color=stimlen)) +
#   geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(paste0("Patient",patient[1]))
#   geom_point(data=fitted_pos_len_summary, aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(paste0("Patient",patient[1]))
```

```
nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
  paste0(NoFragData$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
  "no_fragments_percent_preserved_by_length_pos_wfit.png"),
  plot=nofrag_fitted_len_pos_plot,
  device="png",unit="cm",
  width=15,height=11)
nofrag_fitted_len_pos_plot
```

back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.31 - 1.02"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account
```

```
# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
```

```
table_to_use <- fitted_pos_len_table
# take away column with length information
table_to_use <- table_to_use[,2:ncol(table_to_use)]
```

```
# take averages for positions
```

```
# don't want downward estimates influenced by return upward of U
```

```
# therefore, for downward influence, use only the values before the min
```

```
# take the difference between each value (differences between position proportion correct) **NOTE** pro
```

```
# average the difference in probabilities
```

```
# in case min is close to the end or we are not using a min (for non-U-shaped)
```

```
# functions, we need to get differences _first_ and then average those (e.g.
```

```
# if there is an effect of length and we just average position data,
```

```
# later positions) will have a lower average (because of the length effect)
```

```
# even if all position functions go upward
```

```

table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)

## [1] "mean change in probability for each additional length: "
print(OA_mean_len_diff)

## [1] -0.01500093
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding U-shape): "
print(CurrentLabel)

## [1] "mean change in probability for each additional position (excluding U-shape): "
print(OA_mean_pos_diff)

## [1] -0.05571372
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)

if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
                              2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
  }
}

```

```

    CurrentLabel<-"Average upward change after U minimum"
    print(CurrentLabel)
    print(OA_mean_pos_u_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

    CurrentLabel<-"Proportion of average downward change"
    prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
  }
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}

## [1] "No U-shape in this participant"
if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upward_row])
}

```

```

CurrentLabel<-"return upward value"
print(upward_dist[biggest_return_upward_row])
results_report_DF <- AddReportLine(results_report_DF,
                                   CurrentLabel,
                                   upward_dist[biggest_return_upward_row])

print(" ")
# percentage return upward
percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upward_row]
CurrentLabel <- "Return upward as a proportion of the downward distance:"
print(CurrentLabel)
print(percentage_return_upward)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                                   percentage_return_upward)
}else{
  print("no U-shape in this participant")
}

## [1] "no U-shape in this participant"

FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small sample sizes)
  "preserved ~ stimlen*log_freq",
  "preserved ~ stimlen+log_freq",
  "preserved ~ pos*log_freq",
  "preserved ~ pos+log_freq",
  "preserved ~ stimlen*log_freq + pos*log_freq",
  "preserved ~ stimlen*log_freq + pos",
  "preserved ~ stimlen + pos*log_freq",
  "preserved ~ stimlen + pos + log_freq",
  "preserved ~ (I(pos^2)+pos)*log_freq",
  "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen*log_freq + I(pos^2) + pos",
  "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen + I(pos^2) + pos + log_freq",

  # models without frequency
  "preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",
  "preserved ~ stimlen*pos",
  "preserved ~ I(pos^2)+pos",
  "preserved ~ stimlen + I(pos^2) + pos",
  "preserved ~ stimlen * (I(pos^2) + pos)"
)

FLPRes<-TestModels(FLPModelEquations,PosDat)

## *****
## model index: 21
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##

```

```

## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stimlen:pos
##      2.847382      0.069574      0.033602      -0.406126      0.001867      -0.001867
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3791 Residual
## Null Deviance:      4368
## Residual Deviance: 3830 AIC: 3842
## log likelihood: -1915.226
## Nagelkerke R2: 0.1931346
## % pres/err predicted correctly: -1263.236
## % of predictable range [ (model-null)/(1-null) ]: 0.1391104
## *****
## model index: 11
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      log_freq      I(pos^2)      pos  stimlen:log_freq
##      3.77030      -0.07426      0.21855      0.03498      -0.70338      -0.003498
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3791 Residual
## Null Deviance:      4368
## Residual Deviance: 3831 AIC: 3843
## log likelihood: -1915.426
## Nagelkerke R2: 0.193001
## % pres/err predicted correctly: -1264.194
## % of predictable range [ (model-null)/(1-null) ]: 0.1384575
## *****
## model index: 20
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos
##      3.87402      -0.08311      0.03588      -0.71173
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3793 Residual
## Null Deviance:      4368
## Residual Deviance: 3835 AIC: 3843
## log likelihood: -1917.441
## Nagelkerke R2: 0.1916519
## % pres/err predicted correctly: -1265.45
## % of predictable range [ (model-null)/(1-null) ]: 0.1376027
## *****
## model index: 13
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos      log_freq
##      3.854310      -0.080624      0.035839      -0.711310      0.007982

```

```

##
## Degrees of Freedom: 3796 Total (i.e. Null); 3792 Residual
## Null Deviance: 4368
## Residual Deviance: 3835 AIC: 3845
## log likelihood: -1917.373
## Nagelkerke R2: 0.1916971
## % pres/err predicted correctly: -1265.47
## % of predictable range [ (model-null)/(1-null) ]: 0.1375893
## *****
## model index: 10
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq I(pos^2) pos stim
## 3.793682 -0.075311 0.362759 0.036551 -0.715192
## log_freq:pos
## -0.060019
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3789 Residual
## Null Deviance: 4368
## Residual Deviance: 3829 AIC: 3845
## log likelihood: -1914.422
## Nagelkerke R2: 0.1936719
## % pres/err predicted correctly: -1263.819
## % of predictable range [ (model-null)/(1-null) ]: 0.1387135
## *****
## model index: 12
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos log_freq I(pos
## 3.846372 -0.078352 0.036196 -0.715739 0.129809
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3790 Residual
## Null Deviance: 4368
## Residual Deviance: 3833 AIC: 3847
## log likelihood: -1916.519
## Nagelkerke R2: 0.1922693
## % pres/err predicted correctly: -1265.324
## % of predictable range [ (model-null)/(1-null) ]: 0.1376884
## *****
## model index: 19
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 3.27014 0.03142 -0.69810
##

```

```

## Degrees of Freedom: 3796 Total (i.e. Null); 3794 Residual
## Null Deviance: 4368
## Residual Deviance: 3844 AIC: 3850
## log likelihood: -1921.796
## Nagelkerke R2: 0.1887317
## % pres/err predicted correctly: -1269.173
## % of predictable range [ (model-null)/(1-null) ]: 0.1350676
## *****
## model index: 9
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos log_freq I(pos^2):log_freq
## 3.270723 0.031818 -0.700703 0.154927 0.004385
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3791 Residual
## Null Deviance: 4368
## Residual Deviance: 3840 AIC: 3852
## log likelihood: -1920.101
## Nagelkerke R2: 0.1898691
## % pres/err predicted correctly: -1268.545
## % of predictable range [ (model-null)/(1-null) ]: 0.1354952
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq pos stimlen:log_freq
## 3.02455 -0.05306 0.23790 -0.38514 -0.02900
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3792 Residual
## Null Deviance: 4368
## Residual Deviance: 3847 AIC: 3857
## log likelihood: -1923.573
## Nagelkerke R2: 0.1875386
## % pres/err predicted correctly: -1265.416
## % of predictable range [ (model-null)/(1-null) ]: 0.1376256
## *****
## model index: 17
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos
## 3.11709 -0.06222 -0.38525
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3794 Residual
## Null Deviance: 4368
## Residual Deviance: 3852 AIC: 3858

```

```

## log likelihood: -1926.024
## Nagelkerke R2: 0.1858904
## % pres/err predicted correctly: -1266.877
## % of predictable range [ (model-null)/(1-null) ]: 0.1366311
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq pos stimlen:log_freq log_fr
## 3.025451 -0.052522 0.238541 -0.386311 -0.026782 -0.
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3791 Residual
## Null Deviance: 4368
## Residual Deviance: 3847 AIC: 3859
## log likelihood: -1923.509
## Nagelkerke R2: 0.1875817
## % pres/err predicted correctly: -1265.446
## % of predictable range [ (model-null)/(1-null) ]: 0.1376051
## *****
## model index: 18
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos stimlen:pos
## 3.46950 -0.10490 -0.48421 0.01166
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3793 Residual
## Null Deviance: 4368
## Residual Deviance: 3851 AIC: 3859
## log likelihood: -1925.645
## Nagelkerke R2: 0.1861454
## % pres/err predicted correctly: -1267.405
## % of predictable range [ (model-null)/(1-null) ]: 0.1362715
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos log_freq
## 3.095497 -0.059387 -0.385227 0.009167
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3793 Residual
## Null Deviance: 4368
## Residual Deviance: 3852 AIC: 3860
## log likelihood: -1925.936
## Nagelkerke R2: 0.1859503
## % pres/err predicted correctly: -1266.897

```



```

## % of predictable range [ (model-null)/(1-null) ]: 0.1366173
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos      log_freq pos:log_freq
##      3.07605     -0.05535     -0.38910      0.06568      -0.01264
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3792 Residual
## Null Deviance:      4368
## Residual Deviance: 3850 AIC: 3860
## log likelihood: -1925.172
## Nagelkerke R2: 0.1864636
## % pres/err predicted correctly: -1266.696
## % of predictable range [ (model-null)/(1-null) ]: 0.136754
## *****
## model index: 16
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      2.723      -0.406
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3795 Residual
## Null Deviance:      4368
## Residual Deviance: 3857 AIC: 3861
## log likelihood: -1928.587
## Nagelkerke R2: 0.1841652
## % pres/err predicted correctly: -1269.561
## % of predictable range [ (model-null)/(1-null) ]: 0.1348031
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos      log_freq pos:log_freq
##      2.72227     -0.40691      0.08617      -0.01496
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3793 Residual
## Null Deviance:      4368
## Residual Deviance: 3854 AIC: 3862
## log likelihood: -1927.055
## Nagelkerke R2: 0.1851971
## % pres/err predicted correctly: -1268.869
## % of predictable range [ (model-null)/(1-null) ]: 0.1352742
## *****
## model index: 4

```

```

##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos      log_freq
##      2.71446      -0.40383      0.02001
##
## Degrees of Freedom: 3796 Total (i.e. Null);  3794 Residual
## Null Deviance:      4368
## Residual Deviance: 3856 AIC: 3862
## log likelihood: -1928.137
## Nagelkerke R2:  0.1844684
## % pres/err predicted correctly: -1269.308
## % of predictable range [ (model-null)/(1-null) ]:  0.1349751
## *****
## model index:  1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      log_freq stimlen:log_freq
##      2.98085      -0.25262      0.22975      -0.02762
##
## Degrees of Freedom: 3796 Total (i.e. Null);  3793 Residual
## Null Deviance:      4368
## Residual Deviance: 4232 AIC: 4240
## log likelihood: -2116.095
## Nagelkerke R2:  0.05140159
## % pres/err predicted correctly: -1417.297
## % of predictable range [ (model-null)/(1-null) ]:  0.0342016
## *****
## model index:  15
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      3.0699      -0.2615
##
## Degrees of Freedom: 3796 Total (i.e. Null);  3795 Residual
## Null Deviance:      4368
## Residual Deviance: 4237 AIC: 4241
## log likelihood: -2118.599
## Nagelkerke R2:  0.0495386
## % pres/err predicted correctly: -1418.497
## % of predictable range [ (model-null)/(1-null) ]:  0.03338451
## *****
## model index:  2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)

```

```
##
## Coefficients:
## (Intercept)      stimlen      log_freq
##      3.048156      -0.258678      0.009445
##
## Degrees of Freedom: 3796 Total (i.e. Null);  3794 Residual
## Null Deviance:      4368
## Residual Deviance: 4237  AIC: 4243
## log likelihood:  -2118.493
## Nagelkerke R2:  0.0496179
## % pres/err predicted correctly:  -1418.499
## % of predictable range [ (model-null)/(1-null) ]:  0.03338265
## *****
## model index:  14
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      1.035
##
## Degrees of Freedom: 3796 Total (i.e. Null);  3796 Residual
## Null Deviance:      4368
## Residual Deviance: 4368  AIC: 4370
## log likelihood:  -2183.993
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -1467.523
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****

BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                          AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2

FLPAICSummary <- merge(FLPAICSummary,FLPRes$CoefficientValues,
                      by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                              RunLabel,"_",CurTask,
                              "_freq_len_pos_model_summary.csv"),row.names = FALSE)

kable(FLPAICSummary)
```

Model	AIC Delta	AIC	AICw	NagR	Interstim	log_stimlen	log_freq	log_freq(I(pos^2))	log_freq(I(pos^2) + pos)	log_freq(I(pos^2) + pos) *	log_freq(I(pos^2) + pos) *	len:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	3842.450000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.0018668
							0.4061258					0.0510727
preserved ~ stimlen * log_freq + I(pos^2) + pos	3842.839968978837	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000
							0.0742565	0.0265047	0.0349845	NA	NA	NA
preserved ~ stimlen + I(pos^2) + pos	3842.882075886235	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000
							0.0831078	0.7117347	0.0358834	NA	NA	NA
preserved ~ stimlen + I(pos^2) + pos + log_freq	3844.749554117349	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000
							0.0806243	0.7113099	0.0358391	NA	NA	NA
preserved ~ stimlen * log_freq + (I(pos^2) + pos) *	3844.849329972008	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000
							0.0753106	0.0309198	0.0365541	0.0065774	NA	NA
preserved ~ stimlen + (I(pos^2) + pos) *	3847.438646900932	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000
							0.0783516	0.7150382	0.0360959	0.0070655	NA	NA
preserved ~ stimlen + log_freq	3849.793114028036	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000
							0.6980953	0.0314462	NA	NA	NA	NA
preserved ~ (I(pos^2) + pos) *	3852.273084097630	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000
							0.1549269	0.7000028	0.0316179	0.0038511	NA	NA
preserved ~ stimlen * log_freq + pos	3857.146945600640	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000
							0.0530569	0.0289088	0.03851446	NA	NA	NA
preserved ~ stimlen + pos	3858.151975997410	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000
							0.0622164	0.3852522	NA	NA	NA	NA
preserved ~ stimlen *	3859.168662082517	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000
							0.0525224	0.0267813	0.0040522	NA	NA	NA
preserved ~ stimlen * pos	3859.298395009200	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000
							0.1048953	0.4842097	NA	NA	NA	NA
preserved ~ stimlen + pos + log_freq	3859.871197086065	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000	0.000000000000
							0.0593871	0.3852271	NA	NA	NA	NA

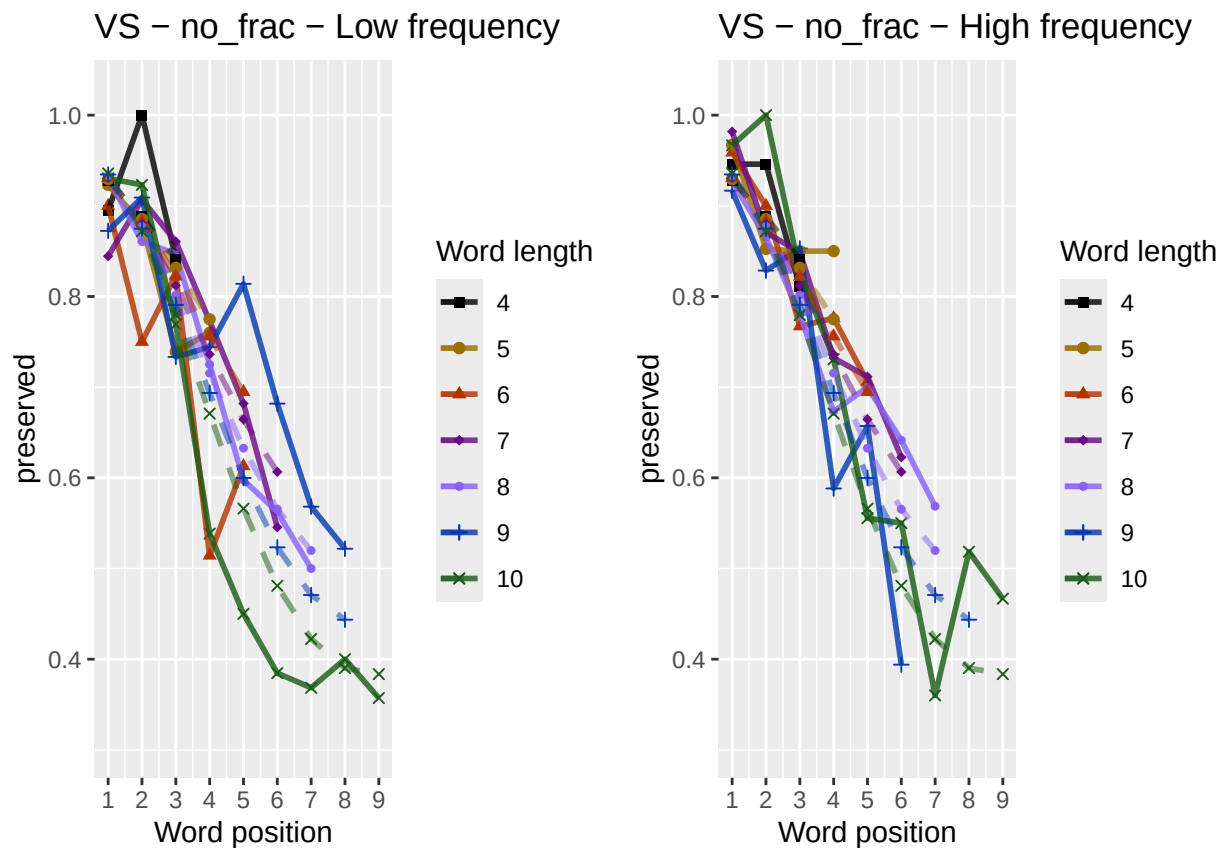

```
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
LF_Plot <- plot_len_pos_obs_predicted(LFdat, paste0(CurPat, " - ", RunLabel, " - Low frequency"), "FLPFitted")

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.

library(ggpubr)
Both_Plots <- ggarrange(LF_Plot, HF_Plot) # labels=c("LF", "HF", ncol=2)

## Warning: Removed 2 rows containing missing values or values outside the scale range (`geom_point()`)
## Warning: Removed 2 rows containing missing values or values outside the scale range (`geom_line()`).

ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_",
              CurTask, "_frequency_effect_length_pos_wfit.png"),
       device="png", unit="cm", width=30, height=11)
print(Both_Plots)
```



```
# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",
  "preserved ~ pos",
  "preserved ~ stimlen",
  "preserved ~ 1"
)
```

```
MERes<-TestModels(MEModelEquations,PosDat)
```

```
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      3.27014      0.03142     -0.69810
##
## Degrees of Freedom: 3796 Total (i.e. Null);  3794 Residual
## Null Deviance:      4368
## Residual Deviance: 3844 AIC: 3850
## log likelihood: -1921.796
## Nagelkerke R2:  0.1887317
## % pres/err predicted correctly: -1269.173
## % of predictable range [ (model-null)/(1-null) ]:  0.1350676
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      2.723      -0.406
##
## Degrees of Freedom: 3796 Total (i.e. Null);  3795 Residual
## Null Deviance:      4368
## Residual Deviance: 3857 AIC: 3861
## log likelihood: -1928.587
## Nagelkerke R2:  0.1841652
## % pres/err predicted correctly: -1269.561
## % of predictable range [ (model-null)/(1-null) ]:  0.1348031
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      1.5096      -0.8425
##
## Degrees of Freedom: 3796 Total (i.e. Null);  3795 Residual
## Null Deviance:      4368
## Residual Deviance: 3931 AIC: 3935
## log likelihood: -1965.539
## Nagelkerke R2:  0.1590289
## % pres/err predicted correctly: -1283.036
## % of predictable range [ (model-null)/(1-null) ]:  0.125627
## *****
```

```

## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.6352      -0.2583
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3795 Residual
## Null Deviance:      4368
## Residual Deviance: 4225 AIC: 4229
## log likelihood: -2112.6
## Nagelkerke R2: 0.05399852
## % pres/err predicted correctly: -1412.628
## % of predictable range [ (model-null)/(1-null) ]: 0.03738091
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      3.0699      -0.2615
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3795 Residual
## Null Deviance:      4368
## Residual Deviance: 4237 AIC: 4241
## log likelihood: -2118.599
## Nagelkerke R2: 0.0495386
## % pres/err predicted correctly: -1418.497
## % of predictable range [ (model-null)/(1-null) ]: 0.03338451
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      1.035
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3796 Residual
## Null Deviance:      4368
## Residual Deviance: 4368 AIC: 4370
## log likelihood: -2183.993
## Nagelkerke R2: 0
## % pres/err predicted correctly: -1467.523
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]

```



```

MEAICSummary<-data.frame(Model=MERes$Model,
                          AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                      by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                              CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ (I(pos^2) + pos)	3849.599	0.00000	1.000000	0.996953	0.188731	17.270144	NA	NA	0.0314162	-	NA
preserved ~ pos	3861.174	11.58165	0.003055	0.003046	0.1218416	52.722653	NA	NA	NA	-	NA
preserved ~ CumErr	3935.078	54.48482	0.000000	0.000000	0.159028	9.509644	NA	-	NA	NA	NA
preserved ~ CumPres	4229.193	379.6064	0.000000	0.000000	0.053998	5.635246	-	NA	NA	NA	NA
preserved ~ stimlen	4241.198	91.60532	0.000000	0.000000	0.049538	3.069940	0.258288	NA	NA	NA	-
preserved ~ 1	4369.987	520.3940	0.000000	0.000000	0.000000	0.035346	NA	NA	NA	NA	NA

```

if(DoSimulations){
  BestMEModelFormulaRnd <- BestMEModelFormula
  if(grepl("CumPres",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
  }else if(grepl("CumErr",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                       family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
  ModelNames<-c(paste0("***",BestMEModelFormula),
                rep(BestMEModelFormulaRnd,RandomSamples))
  AICValues <- c(BestMEModel$aic,RndModelAIC)
  BestMEModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
}

```

```

BestMEMModelRndDF <- BestMEMModelRndDF %>% arrange(AIC)
BestMEMModelRndDF <- rbind(BestMEMModelRndDF,
                           data.frame(Name=c("Random average"),
                                       AIC=c(mean(RndModelAIC))))
BestMEMModelRndDF <- rbind(BestMEMModelRndDF,
                           data.frame(Name=c("Random SD"),
                                       AIC=c(sd(RndModelAIC))))

write.csv(BestMEMModelRndDF,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                 "_best_main_effects_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary, paste0(TablesDir, CurPat, "_",
                                         RunLabel, "_", CurTask,
                                         "_syllable_component_summary.csv"), row.names = FALSE)
kable(syll_component_summary)

```

syll_component	MeanPres	N
1	0.7817460	504
O	0.7360090	1769
P	0.9393939	33
S	0.7863248	234
V	0.7088305	1257

```

# main effects models for data without satellite positions

keep_components = c("0", "V", "1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                             stimlen, stim, pos,
                             preserved, syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data, MEModelEquations)

## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##    3.20666      0.03004     -0.68061
##
## Degrees of Freedom: 3529 Total (i.e. Null); 3527 Residual
## Null Deviance:      4098

```

```

## Residual Deviance: 3616 AIC: 3622
## log likelihood: -1807.769
## Nagelkerke R2: 0.185858
## % pres/err predicted correctly: -1197.097
## % of predictable range [ (model-null)/(1-null) ]: 0.1331809
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## 2.6838 -0.4015
##
## Degrees of Freedom: 3529 Total (i.e. Null); 3528 Residual
## Null Deviance: 4098
## Residual Deviance: 3627 AIC: 3631
## log likelihood: -1813.644
## Nagelkerke R2: 0.1816223
## % pres/err predicted correctly: -1197.26
## % of predictable range [ (model-null)/(1-null) ]: 0.1330626
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr
## 1.4726 -0.8809
##
## Degrees of Freedom: 3529 Total (i.e. Null); 3528 Residual
## Null Deviance: 4098
## Residual Deviance: 3694 AIC: 3698
## log likelihood: -1847.183
## Nagelkerke R2: 0.1571729
## % pres/err predicted correctly: -1209.11
## % of predictable range [ (model-null)/(1-null) ]: 0.1244893
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres
## 1.567 -0.260
##
## Degrees of Freedom: 3529 Total (i.e. Null); 3528 Residual
## Null Deviance: 4098
## Residual Deviance: 3973 AIC: 3977
## log likelihood: -1986.607
## Nagelkerke R2: 0.05040329

```

```
## % pres/err predicted correctly: -1332.921
## % of predictable range [ (model-null)/(1-null) ]: 0.0349127
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      3.032      -0.260
##
## Degrees of Freedom: 3529 Total (i.e. Null); 3528 Residual
## Null Deviance: 4098
## Residual Deviance: 3975 AIC: 3979
## log likelihood: -1987.687
## Nagelkerke R2: 0.04954249
## % pres/err predicted correctly: -1334.742
## % of predictable range [ (model-null)/(1-null) ]: 0.03359513
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      1.009
##
## Degrees of Freedom: 3529 Total (i.e. Null); 3529 Residual
## Null Deviance: 4098
## Residual Deviance: 4098 AIC: 4100
## log likelihood: -2048.785
## Nagelkerke R2: -3.233222e-16
## % pres/err predicted correctly: -1381.176
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
write.csv(SimpSyllMEAICSummary,
          paste0(TablesDir, CurPat, "_", RunLabel, "_",
                  CurTask,
                  "_OV1_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ (I(pos^2) + pos)	3621.538	0.0000001	0.0000000	0.9924276	0.1858580	206658	NA	NA	0.0300362	-	NA
										0.6806074	
preserved ~ pos	3631.289	9.7512840	0.0076302	0.0075724	0.1181622	23683810	NA	NA	NA	-	NA
										0.4014759	
preserved ~ CumErr	3698.366	76.828250	0.0000000	0.0000000	0.1571729	472623	NA	-	NA	NA	NA
								0.8809222			

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumPres	3977.213	55.67587	0.0000000	0.0000000	0.050403	3.566511	-	NA	NA	NA	NA
							0.2600364				
preserved ~ stimlen	3979.374	57.83689	0.0000000	0.0000000	0.049542	3.031668	NA	NA	NA	NA	-
											0.2599972
preserved ~ 1	4099.564	478.03154	0.0000000	0.0000000	0.000000	0.009188	NA	NA	NA	NA	NA

```
# main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
# also reduces data)
```

```
keep_components = c("0", "V")
OVDData <- PosDat[PosDat$syll_component %in% keep_components,]
OVDData <- OVDData %>% select(stim_number,
                             stimlen, stim, pos,
                             preserved, syll_component)
OVDData$CumPres <- CalcCumPres(OVDData)
OVDData$CumErr <- CalcCumErrFromPreserved(OVDData)

SimpSyllMEAICSummary2 <- EvaluateSubsetData(OVDData, MEModelEquations)
```

```
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      3.17213      0.03392     -0.70747
##
## Degrees of Freedom: 3025 Total (i.e. Null); 3023 Residual
## Null Deviance:      3561
## Residual Deviance: 3128 AIC: 3134
## log likelihood: -1564.054
## Nagelkerke R2: 0.1927914
## % pres/err predicted correctly: -1039.736
## % of predictable range [ (model-null)/(1-null) ]: 0.1380225
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      2.6014      -0.3949
##
## Degrees of Freedom: 3025 Total (i.e. Null); 3024 Residual
## Null Deviance:      3561
## Residual Deviance: 3142 AIC: 3146
## log likelihood: -1571.191
## Nagelkerke R2: 0.1868677
```

```

## % pres/err predicted correctly: -1040.805
## % of predictable range [ (model-null)/(1-null) ]: 0.1371376
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      1.3751      -0.9081
##
## Degrees of Freedom: 3025 Total (i.e. Null); 3024 Residual
## Null Deviance:      3561
## Residual Deviance: 3261 AIC: 3265
## log likelihood: -1630.659
## Nagelkerke R2: 0.1364093
## % pres/err predicted correctly: -1076.541
## % of predictable range [ (model-null)/(1-null) ]: 0.1075394
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.650      -0.381
##
## Degrees of Freedom: 3025 Total (i.e. Null); 3024 Residual
## Null Deviance:      3561
## Residual Deviance: 3397 AIC: 3401
## log likelihood: -1698.646
## Nagelkerke R2: 0.07623907
## % pres/err predicted correctly: -1141.538
## % of predictable range [ (model-null)/(1-null) ]: 0.05370657
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      3.0837      -0.2725
##
## Degrees of Freedom: 3025 Total (i.e. Null); 3024 Residual
## Null Deviance:      3561
## Residual Deviance: 3443 AIC: 3447
## log likelihood: -1721.595
## Nagelkerke R2: 0.05531081
## % pres/err predicted correctly: -1160.8
## % of predictable range [ (model-null)/(1-null) ]: 0.0377528
## *****

```

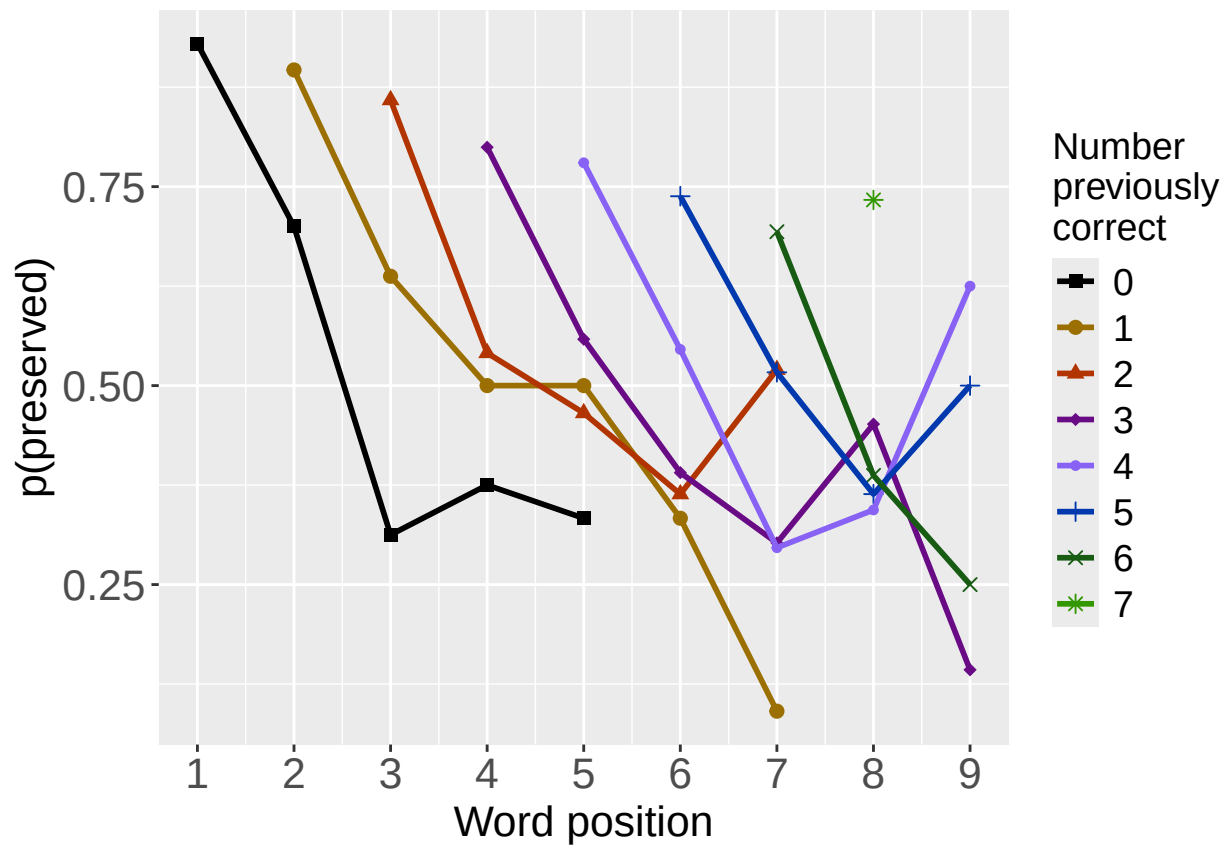
```
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 0.968
##
## Degrees of Freedom: 3025 Total (i.e. Null); 3025 Residual
## Null Deviance: 3561
## Residual Deviance: 3561 AIC: 3563
## log likelihood: -1780.622
## Nagelkerke R2: 1.604921e-16
## % pres/err predicted correctly: -1206.382
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
write.csv(SimpSyllMEAICSummary2,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                 "_OV_main_effects_model_summary.csv"), row.names = FALSE)
kable(SimpSyllMEAICSummary2)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ (I(pos^2) + pos)	3134.108	0.00000	1.0000000	0.9978439	0.1927913	1.1721289	NA	NA	0.0339185	-	NA
										0.7074711	
preserved ~ pos	3146.383	12.27456	0.0021608	0.0021561	0.1868677	0.6014009	NA	NA	NA	-	NA
										0.3949443	
preserved ~ CumErr	3265.318	131.20999	0.0000000	0.0000000	0.1364093	0.3751231	NA	-	NA	NA	NA
								0.908117			
preserved ~ CumPres	3401.292	267.18483	0.0000000	0.0000000	0.0762391	0.6502877	-	NA	NA	NA	NA
							0.381028				
preserved ~ stimlen	3447.193	313.08255	0.0000000	0.0000000	0.0553103	0.0837244	NA	NA	NA	NA	-
											0.2724817
preserved ~ 1	3563.244	429.13621	0.0000000	0.0000000	0.0000000	0.9679921	NA	NA	NA	NA	NA

```
# plot prev err and prev cor plots
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

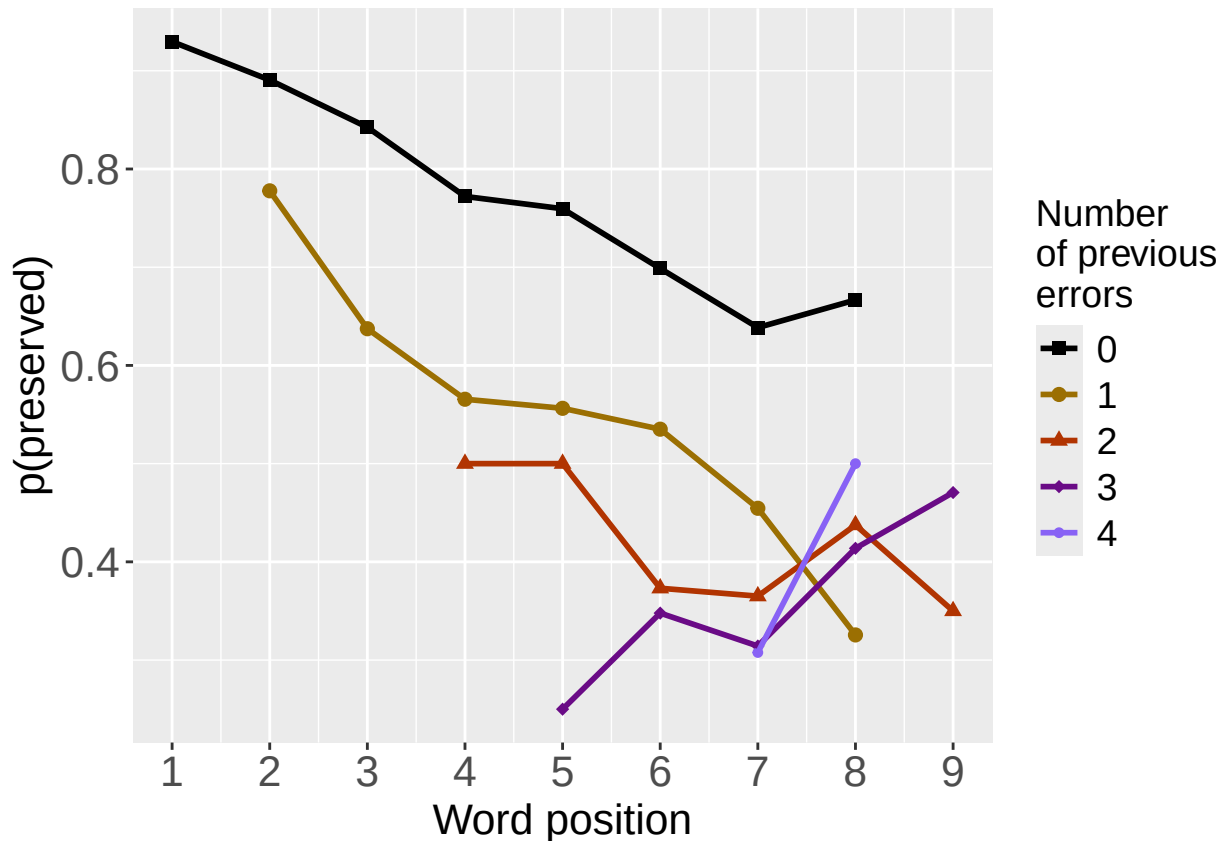
```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```

```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

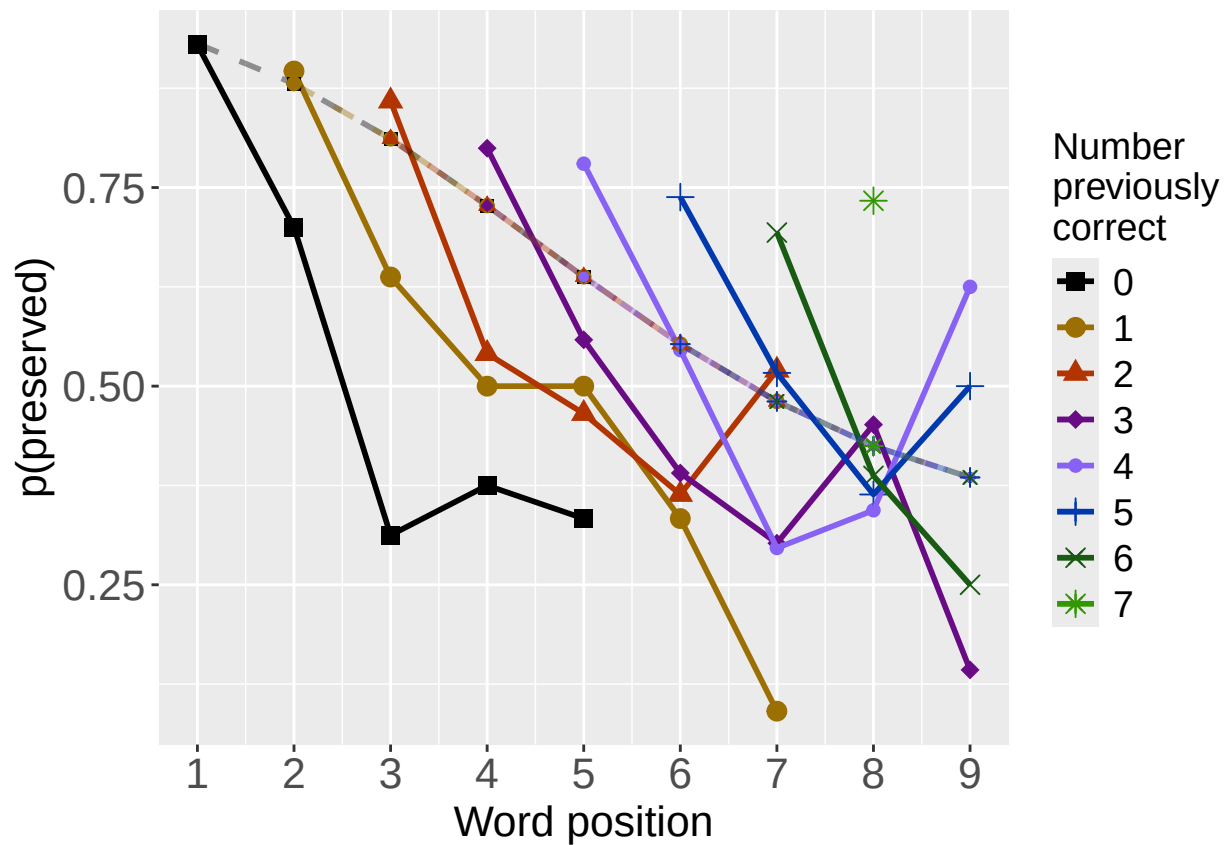
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)

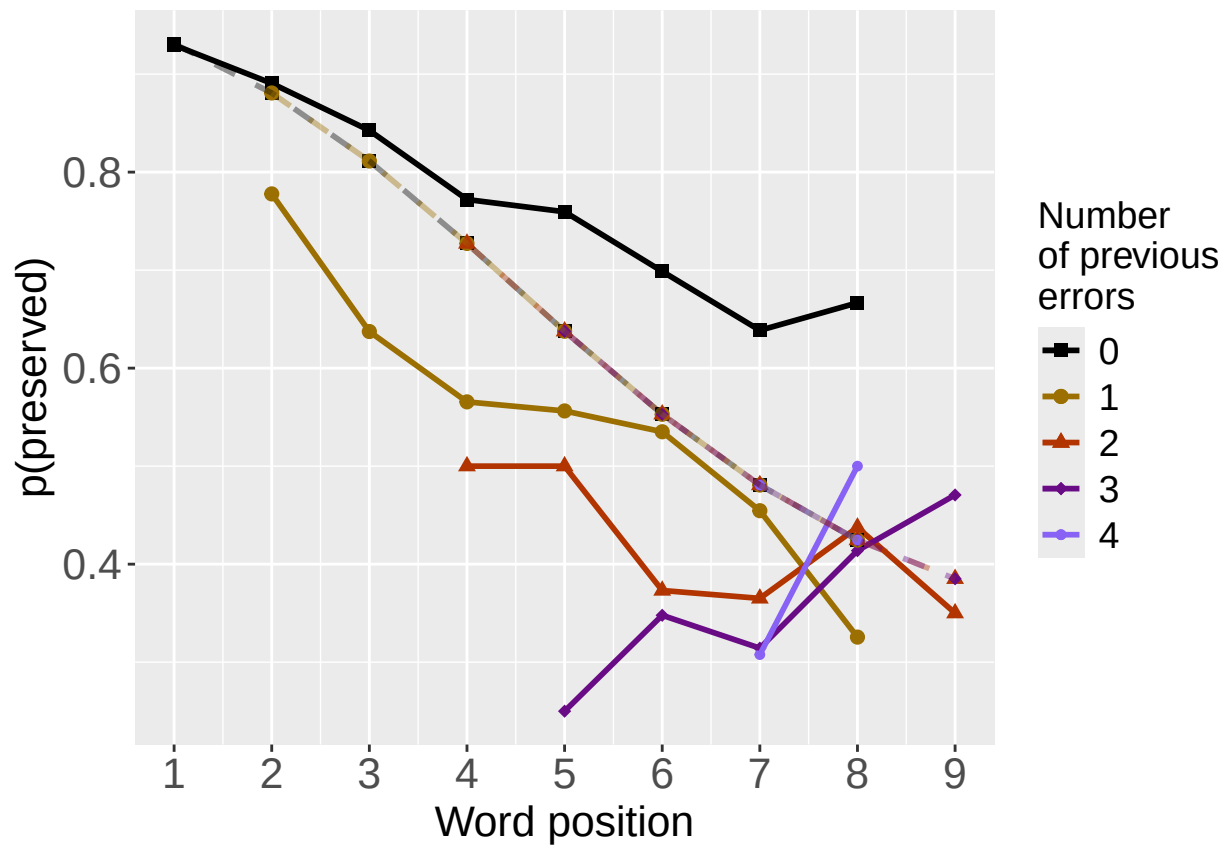
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)

```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

```

```

CumAICSummary <- NULL

#####
# level 2 -- Add position squared (quadratic with position)
#####

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}

```

```
#####
# level 2 -- Add length
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos stimlen
## 3.87402 0.03588 -0.71173 -0.08311
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3793 Residual
## Null Deviance: 4368
## Residual Deviance: 3835 AIC: 3843
## log likelihood: -1917.441
## Nagelkerke R2: 0.1916519
## % pres/err predicted correctly: -1265.45
## % of predictable range [ (model-null)/(1-null) ]: 0.1376027
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 3.27014 0.03142 -0.69810
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3794 Residual
## Null Deviance: 4368
## Residual Deviance: 3844 AIC: 3850
## log likelihood: -1921.796
## Nagelkerke R2: 0.1887317
## % pres/err predicted correctly: -1269.173
## % of predictable range [ (model-null)/(1-null) ]: 0.1350676
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen
##      3.0699      -0.2615
##
## Degrees of Freedom: 3796 Total (i.e. Null);  3795 Residual
## Null Deviance:      4368
## Residual Deviance: 4237  AIC: 4241
## log likelihood:  -2118.599
## Nagelkerke R2:  0.0495386
## % pres/err predicted correctly:  -1418.497
## % of predictable range [ (model-null)/(1-null) ]:  0.03338451
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	I(pos^2)	pos	stimlen
preserved ~ (I(pos^2) + pos) + stimlen	3842.8820	0.000000	1.000000	0.966278	0.191651	3.874024	0.0358834	-	-
preserved ~ (I(pos^2) + pos)	3849.5936	7.710644	0.034898	0.033721	0.188731	3.270144	0.0314162	-	NA
preserved ~ stimlen	4241.1983	398.315968	0.000000	0.000000	0.049538	3.069940	NA	NA	-
									0.2615473

```
#####
# level 2 -- add cumulative preserved
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos      CumPres
##      3.80455      0.05569     -1.24267      0.49529
##
## Degrees of Freedom: 3796 Total (i.e. Null);  3793 Residual
## Null Deviance:      4368
## Residual Deviance: 3692  AIC: 3700
## log likelihood:  -1846.068
## Nagelkerke R2:  0.2385635
## % pres/err predicted correctly:  -1201.475
## % of predictable range [ (model-null)/(1-null) ]:  0.1811667
## *****
## model index:  1
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      3.27014      0.03142     -0.69810
##
## Degrees of Freedom: 3796 Total (i.e. Null);  3794 Residual
## Null Deviance:      4368
## Residual Deviance: 3844 AIC: 3850
## log likelihood: -1921.796
## Nagelkerke R2:  0.1887317
## % pres/err predicted correctly: -1269.173
## % of predictable range [ (model-null)/(1-null) ]:  0.1350676
## *****
## model index:  3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.6352      -0.2583
##
## Degrees of Freedom: 3796 Total (i.e. Null);  3795 Residual
## Null Deviance:      4368
## Residual Deviance: 4225 AIC: 4229
## log likelihood: -2112.6
## Nagelkerke R2:  0.05399852
## % pres/err predicted correctly: -1412.628
## % of predictable range [ (model-null)/(1-null) ]:  0.03738091
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	I(pos^2)	pos	CumPres
preserved ~ (I(pos^2) + pos) + CumPres	3700.136	0.0000	1	1	0.2385635	3.804546	0.0556922	- 1.2426722	0.4952891
preserved ~ (I(pos^2) + pos)	3849.593	149.4562	0	0	0.1887317	3.270144	0.0314162	- 0.6980953	NA
preserved ~ CumPres	4229.199	529.0627	0	0	0.0539985	1.635246	NA	NA	- 0.2582888

```
#####
# level 2 -- Add linear position (NOT quadratic)
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```

CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary,paste0(TablesDir,CurPat,"_",
                               RunLabel,"_",CurTask,
                               "_main_effects_plus_one_model_summary.csv"),row.names = FALSE)
kable(CumAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	(pos^2)	pos	stimlen	CumPres
preserved ~ (I(pos^2)	3700.136	0.000000	1.000000	0.000000	0.238563	3.804546	0.0556922	-	NA	0.4952891
+ pos) + CumPres							1.2426722			
preserved ~ (I(pos^2)	3842.882	0.000000	1.000000	0.966278	0.191651	3.874024	0.0358834	-	-	NA
+ pos) + stimlen							0.7117347		0.0831078	
preserved ~ (I(pos^2)	3849.593	0.710644	0.034898	0.033721	0.188731	3.270144	0.0314162	-	NA	NA
+ pos)							0.6980953			
preserved ~ (I(pos^2)	3849.593	0.710644	0.034898	0.033721	0.188731	3.270144	0.0314162	-	NA	NA
+ pos)							0.6980953			
preserved ~ CumPres	4229.195	29.062695	0.000000	0.000000	0.053998	5.635246	NA	NA	NA	-
										0.2582888
preserved ~ stimlen	4241.193	98.315968	0.000000	0.000000	0.049538	3.069940	NA	NA	-	NA
									0.2615473	
NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA

```

# explore influence of frequency and length

if(grepl("stimlen",BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + log_freq")
  )
}else if(grepl("log_freq",BestModelFormulaL2)){ # if log_freq is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + stimlen")
  )
}else{
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + log_freq"),
    paste0(BestModelFormulaL2," + stimlen"),
    paste0(BestModelFormulaL2," + stimlen + log_freq")
  )
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)

```

```

## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```



```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos      CumPres      stimlen
##      4.29716      0.05919      -1.24922      0.49120      -0.06864
##
## Degrees of Freedom: 3796 Total (i.e. Null);  3792 Residual
## Null Deviance:      4368
## Residual Deviance: 3686 AIC: 3696
## log likelihood:  -1843.195
## Nagelkerke R2:  0.2404154
## % pres/err predicted correctly:  -1199.135
## % of predictable range [ (model-null)/(1-null) ]:  0.1827599
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos      CumPres      stimlen      log_freq
##      4.26877      0.05915      -1.24897      0.49148      -0.06499      0.01190
##
## Degrees of Freedom: 3796 Total (i.e. Null);  3791 Residual
## Null Deviance:      4368
## Residual Deviance: 3686 AIC: 3698
## log likelihood:  -1843.052
## Nagelkerke R2:  0.2405072
## % pres/err predicted correctly:  -1199.187
## % of predictable range [ (model-null)/(1-null) ]:  0.1827251
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos      CumPres
##      3.80455      0.05569      -1.24267      0.49529
##
## Degrees of Freedom: 3796 Total (i.e. Null);  3793 Residual
## Null Deviance:      4368
## Residual Deviance: 3692 AIC: 3700
## log likelihood:  -1846.068
## Nagelkerke R2:  0.2385635
## % pres/err predicted correctly:  -1201.475
## % of predictable range [ (model-null)/(1-null) ]:  0.1811667
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:

```

```
## (Intercept)      I(pos^2)      pos      CumPres      log_freq
##      3.79982      0.05597     -1.24279     0.49542     0.02338
##
## Degrees of Freedom: 3796 Total (i.e. Null);  3792 Residual
## Null Deviance:      4368
## Residual Deviance: 3691 AIC: 3701
## log likelihood:  -1845.486
## Nagelkerke R2:  0.2389392
## % pres/err predicted correctly:  -1201.302
## % of predictable range [ (model-null)/(1-null) ]:  0.1812845
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      1.035
##
## Degrees of Freedom: 3796 Total (i.e. Null);  3796 Residual
## Null Deviance:      4368
## Residual Deviance: 4368 AIC: 4370
## log likelihood:  -2183.993
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -1467.523
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
```

```
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

AICSummary<-data.frame(Model=Level3Res$Model,
                        AIC=Level3Res$AIC,
                        row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                    by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                           CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	I(pos^2)	pos	CumPres	log_freq	stimlen
preserved ~ (I(pos^2) + pos) + CumPres + stimlen	3696.39	0.000000	1.000000	0.595570	0.240414	1.249220	0.0591905	-	0.491198	NA	-
											0.0686401

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	$\mu(\text{pos}^2)$	pos	CumPres	log_freq	stimlen
preserved ~ (I(pos^2) + pos) + CumPres + stimlen + log_freq	3698.105	5714872	2.424248	2526700	0.240504	2687700	0.0591459	-	0.491478	0.119039	-
								1.248973			0.0649915
preserved ~ (I(pos^2) + pos) + CumPres	3700.136	746586	0.153617	0091489	0.2385635	045460	0.0556922	-	0.495289	NA	NA
								1.242672			
preserved ~ (I(pos^2) + pos) + CumPres + log_freq	3700.971	581390	0.101196	0160269	0.2389397	998240	0.0559653	-	0.495416	0.233798	NA
								1.242787			
preserved ~ 1	4369.987	3.596882	0.000000	0000000	0.0000000	0000000	0.035346	NA	NA	NA	NA

```
# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[BestModelIndexL3]
```

```
BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions
```

```
## Single term deletions
##
## Model:
## preserved ~ (I(pos^2) + pos) + CumPres + stimlen
##           Df Deviance      AIC
## pos       1   3872.9 3880.9
## CumPres   1   3834.9 3842.9
## I(pos^2)  1   3728.9 3736.9
## stimlen   1   3692.1 3700.1
## <none>     3686.4 3696.4
```

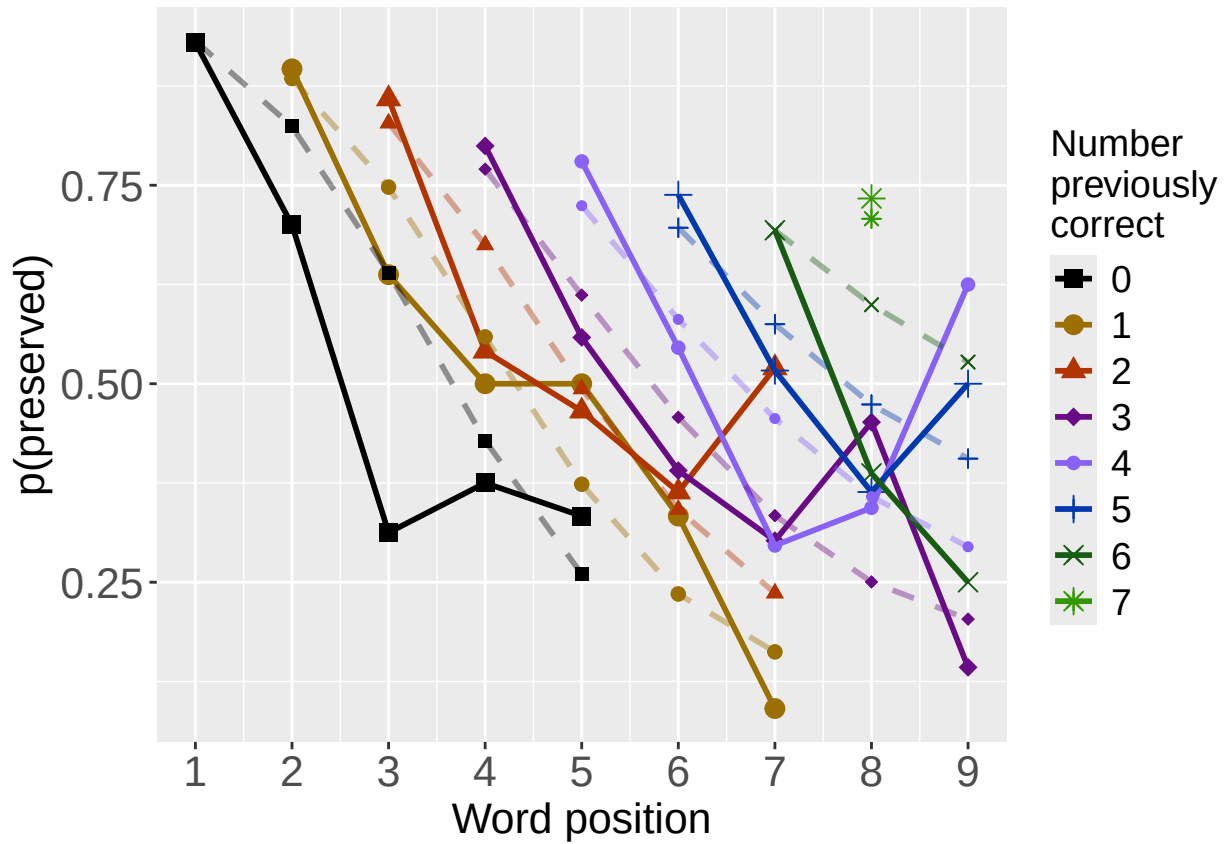
```
#####  
# Single deletions from best model  
#####  
  
write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",  
                                     RunLabel,"_",CurTask,  
                                     "_best_model_single_term_deletions.csv"),  
          row.names = TRUE)
```

```
# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)
```

```
PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

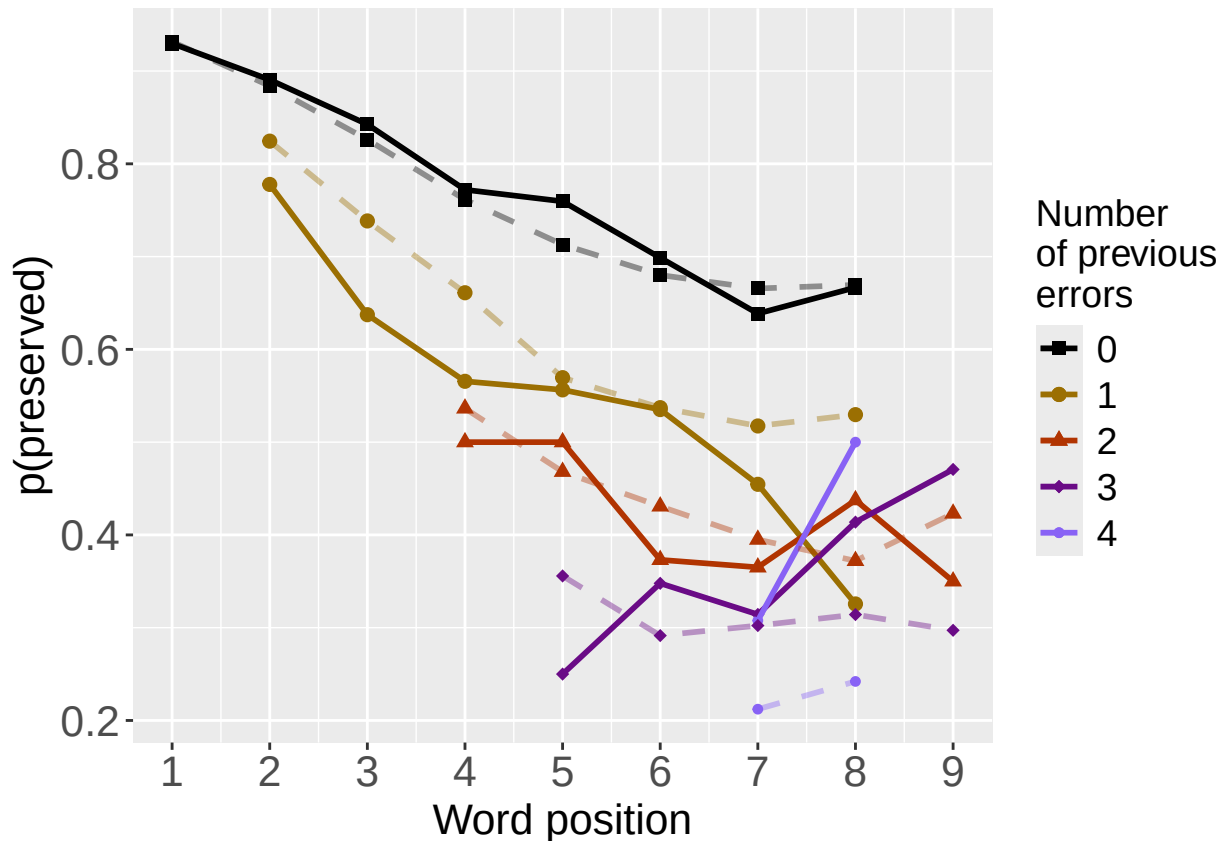
```
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",
  RunLabel,"_",CurTask,
  "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression =

## Saving 6.5 x 4.5 in image
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "

## Saving 6.5 x 4.5 in image
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
      family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
}

```

```

}
ModelNames<-c(paste0("***",BestModelFormulaL3),
              rep(BestModelFormulaL3Rnd,RandomSamples))
AICValues <- c(BestModelL3$aic,RndModelAIC)
BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random average"),
                                   AIC=c(mean(RndModelAIC))))
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random SD"),
                                   AIC=c(sd(RndModelAIC))))
write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_best_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##          2.723         -0.406
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3795 Residual
## Null Deviance: 4368
## Residual Deviance: 3857 AIC: 3861
## log likelihood: -1928.587
## Nagelkerke R2: 0.1841652
## % pres/err predicted correctly: -1269.561
## % of predictable range [ (model-null)/(1-null) ]: 0.1348031

```

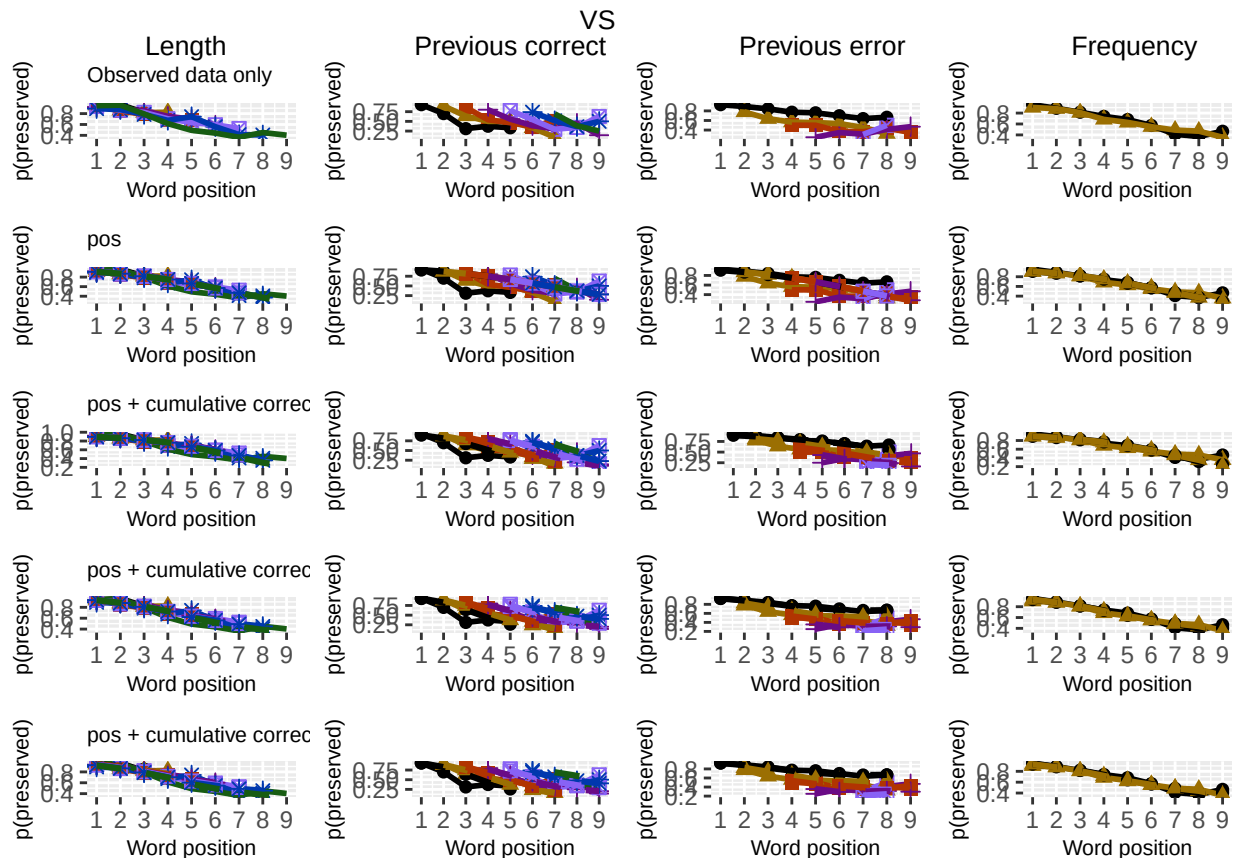
```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos      CumPres
##      2.8576      -0.7023      0.4491
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3794 Residual
## Null Deviance:      4368
## Residual Deviance: 3731 AIC: 3737
## log likelihood: -1865.4
## Nagelkerke R2: 0.2260306
## % pres/err predicted correctly: -1209.764
## % of predictable range [ (model-null)/(1-null) ]: 0.1755226
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos      CumPres      I(pos^2)
##      3.80455      -1.24267      0.49529      0.05569
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3793 Residual
## Null Deviance:      4368
## Residual Deviance: 3692 AIC: 3700
## log likelihood: -1846.068
## Nagelkerke R2: 0.2385635
## % pres/err predicted correctly: -1201.475
## % of predictable range [ (model-null)/(1-null) ]: 0.1811667
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos      CumPres      I(pos^2)      stimlen
##      4.29716      -1.24922      0.49120      0.05919      -0.06864
##
## Degrees of Freedom: 3796 Total (i.e. Null); 3792 Residual
## Null Deviance:      4368
## Residual Deviance: 3686 AIC: 3696
## log likelihood: -1843.195
## Nagelkerke R2: 0.2404154
## % pres/err predicted correctly: -1199.135
## % of predictable range [ (model-null)/(1-null) ]: 0.1827599
## *****
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```



```
## i you have requested 8 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes c
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes c
## i you have requested 8 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
ggsave(paste0(PlotName, ".tif"), plot=FactorPlot, width = 360, height=400, units="mm", device="tiff", compress=
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot
```



```
DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
CurTask,"_dominance_analysis_table.csv"),
row.names = TRUE)
kable(DAContributionAverage)
```

	I(pos ²)	pos	CumPres	stimlen
McFadden	0.0476141	0.0712721	0.0274932	0.0096643

	I(pos ²)	pos	CumPres	stimlen
SquaredCorrelation	0.0507448	0.0743503	0.0284582	0.0107665
Nagelkerke	0.0507448	0.0743503	0.0284582	0.0107665
Estrella	0.0541992	0.0808790	0.0311561	0.0110684

	deviance	deviance_explained
pos + CumPres + I(pos^2) + stimlen	3686.390	681.5969
pos + CumPres + I(pos^2)	3692.136	675.8503
pos + CumPres	3730.801	637.1860
pos	3857.174	510.8124
null	4367.987	0.0000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##                                model deviance deviance_explained percent_explained
## pos + CumPres + I(pos^2) + stimlen pos + CumPres + I(pos^2) + stimlen 3686.390      681.5969      15.60437
## pos + CumPres + I(pos^2)                pos + CumPres + I(pos^2) 3692.136      675.8503      15.47281
## pos + CumPres                pos + CumPres 3730.801      637.1860      14.58763
## pos                pos 3857.174      510.8124      11.69446
## null                null 4367.987      0.0000      0.00000
##                                percent_of_explained_deviance increment_in_explained
## pos + CumPres + I(pos^2) + stimlen 100.00000      0.8431063
## pos + CumPres + I(pos^2)                99.15689      5.6726095
## pos + CumPres                93.48428      18.5408077
## pos                74.94348      74.9434765
## null                NA      0.0000000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
```

	percent_explained	percent_of_explained_deviance	increment_in_explained
pos + CumPres + I(pos ²) + stimlen	15.60437	100.00000	0.8431063
pos + CumPres + I(pos ²)	15.47281	99.15689	5.6726095
pos + CumPres	14.58763	93.48428	18.5408077
pos	11.69446	74.94348	74.9434765
null	0.00000	NA	0.0000000

```

                                "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents

```

```

##           Nagelkerke
## I(pos^2)  0.3088170
## pos      0.4524732
## CumPres  0.1731881
## stimlen  0.0655217

```

```

sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)

```

69

```

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

```

```

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

sse_table<-sse_results_table(sse_results_list)
write.csv(sse_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                          CurTask,"_sse_results_table.csv"),row.names = TRUE)

sse_table

##               model p_accounted_for model_deviance  diff_pos diff_pos+CumPres diff_pos+CumPres+I(pos^2)
## 1             preserved ~ pos          0.8282750      3857.174 0.00000000      -0.08234963      -0.113871298
## 2      preserved ~ pos+CumPres          0.9106247      3730.801 0.08234963       0.00000000      -0.031521672
## 3      preserved ~ pos+CumPres+I(pos^2)          0.9421463      3692.136 0.11387130       0.03152167       0.000000000

```

model	p_accounted_for	model_deviance
preserved ~ pos	0.8282750	3857.174
preserved ~ pos+CumPres	0.9106247	3730.801
preserved ~ pos+CumPres+I(pos^2)	0.9421463	3692.136
preserved ~ pos+CumPres+I(pos^2)+stimlen	0.9437953	3686.390

model	diff_pos	diff_pos+CumPres	diff_pos+CumPres+I(pos^2)	diff_pos+CumPres+I(pos^2)+stimlen
preserved ~ pos	0.0000000	-0.0823496	-0.1138713	-0.1155203
preserved ~ pos+CumPres	0.0823496	0.0000000	-0.0315217	-0.0331706
preserved ~ pos+CumPres+I(pos^2)	0.1138713	0.0315217	0.0000000	-0.0016490
preserved ~ pos+CumPres+I(pos^2)+stimlen	0.1155203	0.0331706	0.0016490	0.0000000

```
## 4 preserved ~ pos+CumPres+I(pos^2)+stimlen      0.9437953      3686.390 0.11552026      0.03317064      0.001648965
##   diff_pos+CumPres+I(pos^2)+stimlen
## 1      -0.115520263
## 2      -0.033170636
## 3      -0.001648965
## 4      0.000000000
```

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF, paste0(TablesDir, CurPat, "_", RunLabel, "_",
                                   CurTask, "_results_report_df.csv"), row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```